

Blueprint: Telecom Network Coverage Analytics & Regional Clustering (India)

0. One-Line Pitch

A data analytics + ML project that takes real Indian cell tower data, engineers regional coverage features, and uses clustering and descriptive analytics to identify well-served, underserved, and coverage-gap regions — presented as an interactive map-based dashboard.

Positioning note: this project targets ML/data-analyst internships (EDA, feature engineering, unsupervised ML, geospatial analysis, dashboarding). No Spark/Hadoop-scale tooling is required or claimed — pandas/scikit-learn is the right tool for this data volume.

1. Core Question

Given the locations, radio types, and carriers of cell towers across India, can we identify which regions are well-covered, which are underserved, and how coverage differs by carrier and network generation (2G/3G/4G/5G) — surfacing this as an actionable regional coverage map?

2. Dataset Sources

Data	Source	Access	Notes
Mobile Network Coverage India (404.csv, 405.csv)	Kaggle (originally sourced from OpenCellID)	Direct CSV download, free Kaggle account, no API/rate limits	Primary tower-level data — two files because India has two MCC codes (404, 405); concatenate them into one table
MCC-MNC India.csv	Same Kaggle dataset	Bundled with the above	Carrier lookup table — decodes MNC codes into carrier names (Jio, Airtel, Vi, BSNL, etc.)
states_india.geojson	Same Kaggle dataset	Bundled with the above	State-level boundary polygons — used for Map 1 (state granularity)
District boundary shapefile/GeoJSON	DataMeet/ LGD-sourced, community-maintained (freely licensed CC0/CC-BY)	Free download, separate from the Kaggle dataset	Used for Map 2 (district granularity) — confirm district-name field matches cleanly before committing

Two-granularity approach: this project now produces **two maps**, built via the same underlying pipeline:

- **Map 1 (state-level):** uses the built-in `states_india.geojson`,

36 regions — build this first, it's simpler and gets you a fully working end-to-end pipeline faster.

- **Map 2 (district-level):** uses the external district boundary file, 700+ regions — once Map 1's pipeline works, swap in the district boundary file and re-run the same aggregation/clustering code. This is mostly a "change the join file" step, not a rebuild, since the cleaning/feature engineering/clustering logic is shared.
- Dashboard should offer a toggle between the two views.

Confirmed columns (from official Kaggle dataset description):

Column	Meaning	Reliability
Radio	Network generation (2G/3G/4G/5G)	Reliable — use directly
MCC	Mobile Country Code	Reliable — will be constant (India)
MNC	Mobile Network Code (identifies carrier)	Reliable — decode via lookup table
LAC/TAC/NID	Location Area Code	Reliable
CID	Unique tower/sector identifier	Reliable
Longitude / Latitude	Tower coordinates	Reliable — core of the dataset
Range	Approximate coverage radius (meters)	Reliable — good density/coverage-area proxy
Samples	Number of measurements behind this data point	Reliable — usable as a confidence weight

Changeable	1 = location derived from crowd-processed samples, 0 = obtained directly from the telecom firm	Reliable — useful data-quality/confidence flag
Created / Updated	UNIX timestamps	Reliable — usable for recency analysis
AverageSignal	Average signal strength	Not reliable — confirmed mostly 0/missing. OpenCellID's own documentation states most values are 0 because signal strength was often never received from contributors. Do not build core features around this column.

Confirmed action item (already resolved): the dataset description explicitly confirms AverageSignal is mostly zero/unpopulated — this is a structural data quality issue, not something to impute around. The blueprint below has been updated to exclude it from core feature engineering. Still worth a `.isnull().sum() / value-count` check on the actual file to confirm the real non-zero percentage, and to check row count/file size for planning purposes.

3. Key Features to Engineer

Density & Distribution

- Tower count per district/region (after reverse-geocoding lat/long)
- Tower density per unit area (towers per sq. km, if area data is added)
- Radio-type mix per region (% 2G vs 3G vs 4G vs 5G towers)

Carrier-Level

- Tower count per carrier (via MNC) per region
- Carrier presence — how many distinct carriers operate in a given region (competition indicator)

Coverage Quality Proxy

- Average signal range per region (Range column — reliable)
- Average sample count per region (Samples column — proxy for data confidence/tower activity level)
- % of "directly obtained from telecom firm" towers per region (Changeable=0 ratio — a genuine data-quality/authority signal, and also a good talking point since it distinguishes verified vs. crowd-estimated coverage)
- ~~Average signal strength~~ — excluded; confirmed mostly 0/unpopulated in the source data (see Section 2)

Derived Regional Labels

- Urban vs. rural proxy (using tower density as a stand-in, since dense areas typically indicate urban zones)
- Coverage tier classification (well-served / moderately-served / underserved) — this can be your clustering *output*, or a rule-based label to validate clusters against

4. Recommended Models & Techniques

Task	Method	Purpose
------	--------	---------

Regional coverage clustering	K-Means	Primary technique — group regions into coverage tiers based on density, radio-mix, carrier presence
Cluster validation	DBSCAN (compare against K-Means)	DBSCAN handles irregular/non-spherical clusters and naturally flags sparse "gap" zones as noise/outliers — useful contrast to K-Means
Optimal cluster count	Elbow method + Silhouette score	Justify your choice of K rigorously, not arbitrarily
Descriptive analytics	Groupby aggregations, pivot tables	Carrier comparison, radio-generation distribution by state
Geospatial visualization	Choropleth/scatter map (Plotly/Folium)	Core visual deliverable
Optional stretch (adds supervised ML breadth to your resume)	Classification model (Random Forest) predicting coverage tier from regional features	Turns your rule-based/cluster-derived labels into a supervised task — validates whether coverage tier is predictable from a smaller feature set, and adds a second ML paradigm to your project

Why this model selection is defensible: clustering is the right primary tool because you're discovering structure (coverage tiers) rather than predicting a pre-existing labeled outcome — there's no ground-truth "coverage tier" label handed to you, so unsupervised learning is the honest, correct approach, not a shortcut.

5. Step-by-Step Blueprint

Phase 1 — Data Collection & Initial Inspection

1. Download the Kaggle Mobile Network Coverage India dataset.
2. Load into pandas; run `.info()`, `.describe()`, `.isnull().sum()` to confirm actual size, column completeness, and missing-value extent.
3. Decide scope: full India, or a manageable multi-state subset if the file is unexpectedly large.

Phase 2 — Data Cleaning

1. Handle missing/invalid latitude-longitude values (drop or flag rows outside India's bounding box).
2. Deduplicate towers if duplicate Cell ID/location entries exist.
3. Decode MNC values into carrier names using a manual lookup table (India's MNC-to-carrier mapping is publicly documented and small — build this once).
4. Decode radio type field into a clean categorical (2G/3G/4G/5G).
5. Drop or ignore AverageSignal as a feature (confirmed mostly 0/unpopulated); confirm Range and Samples are well-populated before relying on them.

Phase 3 — Regional Enrichment

1. Reverse-geocode latitude/longitude to **state** using `states_india.geojson` + a spatial join (`geopandas`) — no paid geocoding API needed.
2. Aggregate tower-level data up to state level: tower count, radio-type mix, carrier count, coverage metrics per state.
3. Once the state-level pipeline (Phases 3–8) works end-to-end, repeat step 1 using the **district** boundary file instead, and re-aggregate at district level. Reuse the same feature engineering and clustering code — only the join file and grouping key change.

Phase 4 — Feature Engineering

1. Compute tower density (towers per district, normalized if area data

- is available).
2. Compute radio-generation mix percentages per district.
 3. Compute carrier diversity (number of distinct carriers present) per district.
 4. Compute average signal range/strength per district (if data quality supports it).
 5. Construct a rule-based coverage tier label (well-served / moderate / underserved) as a sanity-check reference — not the primary target, but useful for validating your clusters.

Phase 5 — Exploratory Data Analysis

1. Map tower density across India (state and district level).
2. Distribution of radio types nationally and regionally — visualize the 2G-to-5G transition geographically.
3. Carrier comparison — which carriers dominate which regions.
4. Identify visibly sparse/underserved regions before any modeling, as a baseline expectation to check your clustering against later.

Phase 6 — Clustering

1. Select features for clustering (density, radio-mix percentages, carrier diversity, signal metrics).
2. Scale features (StandardScaler) before clustering.
3. Run K-Means with a range of K values; use elbow method and silhouette score to select final K.
4. Run DBSCAN as a comparison/validation method; note which regions it flags as outliers/gaps.
5. Interpret and label each cluster meaningfully (e.g., "Cluster 1: Dense 4G/5G urban," "Cluster 2: Sparse 2G/3G rural gap zones").
6. Validate clusters against your rule-based coverage tier labels from Phase 4 — do they broadly agree? Explain any interesting mismatches.

Phase 7 — Optional Supervised Extension

1. Using cluster labels (or rule-based tier labels) as the target, train a

- Random Forest classifier to predict coverage tier from a smaller feature set.
2. Evaluate with accuracy, precision/recall per class, and confusion matrix.
 3. Use SHAP or feature importance to explain which factors most influence coverage tier classification.

Phase 8 — Presentation Layer (Dashboard)

1. Build a Streamlit dashboard with:
 - **Toggle between State-level and District-level map views** (both colored by coverage cluster/tier)
 - Interactive India map colored by coverage cluster/tier, click-to-inspect region stats
 - Carrier comparison charts (tower count, radio-mix by carrier)
 - Radio-generation distribution over regions
 - Cluster summary panel (what defines each cluster)
 - If Phase 7 completed: classifier prediction interface + feature importance panel
-

6. Team Split (2 people)

Person 1 — Data Engineering & Feature Layer

- Data cleaning, MNC/carrier decoding, reverse-geocoding, district-level aggregation, feature engineering, EDA.

Person 2 — Modeling & Presentation Layer

- Clustering (K-Means, DBSCAN), cluster validation and interpretation, optional supervised extension, Streamlit dashboard build.

Both should collaborate on EDA and final report/viva prep.

7. Viva / Interview Narrative

Why this project is relevant: It demonstrates real geospatial data engineering (reverse-geocoding, regional aggregation), rigorous unsupervised ML (clustering with proper validation via elbow/silhouette methods, not an arbitrarily chosen K), and a genuinely useful business question — coverage gap identification — that maps directly onto what a telecom company's network planning team would care about.

Likely questions to prepare for:

1. "Why clustering instead of a supervised model?" — explain there's no pre-existing ground-truth coverage-tier label in the raw data; clustering is the honest way to discover structure, and the optional supervised extension (Phase 7) shows you understand both paradigms and when each applies.
2. "How did you choose the number of clusters?" — walk through elbow method and silhouette score results, not a guessed number.
3. "How did you validate your clusters actually mean something?" — describe the cross-check against your rule-based coverage tier labels and any real-world sanity checks (e.g., known urban vs. rural areas).
4. "Why K-Means and not just DBSCAN, or vice versa?" — K-Means gives clean, interpretable groupings; DBSCAN is better at flagging irregular sparse gap zones as outliers — using both and comparing is a stronger answer than picking one blindly.
5. "How reliable is OpenCellID data?" — be upfront: it's crowd-sourced tower location data, so there may be reporting gaps or inaccuracies in less-monitored regions; state this as a known limitation in your report, not something to hide.

8. Explicit Scope Boundaries (state clearly in your report)

- Coverage tiers are **relative, data-driven groupings**, not official regulatory or carrier-provided coverage assessments.
- OpenCellID data is **crowd-sourced**, so regional data density itself may reflect app/user activity levels, not only actual tower presence — a real, worth-acknowledging limitation.
- This is a **batch, offline analytical project**, not a live coverage-monitoring system.